

Konekt

RNCP 37873 · CDA · Niveau 6

Support de soutenance

Réseau social Konekt

Une plateforme sociale full-stack temps réel — Next.js 15, Prisma, PostgreSQL, SSE

Christophe Lecart

Concepteur Développeur d'Applications · Promotion 2024-2026

Zone01 Rouen Normandie

Soutenance 2026

Sommaire

01

Le candidat & le contexte

Parcours, reconversion, formation

02

Le projet Konekt

Problématique, périmètre, équipe

03

Conception & modélisation

User stories, MCD, design system

04

Architecture & développement

Stack, API, temps réel, sécurité

05

Tests, CI/CD & déploiement

Qualité, automatisation, production

06

Bilan & compétences RNCP

Apprentissages, mapping des blocs

Christophe Lecart



RECONVERSION DEV

Du terrain industriel au code

Technicien industriel pendant 20 ans (KONE Ascenseurs, SOCOTEC Équipements), j'ai entamé en 2024 une transition complète vers le développement full-stack via la formation Zone01 Rouen Normandie.

20 ans

Expérience
industrielle

2024

Entrée
Zone01

Niveau 6

Certification
visée (RNCP)

Full-stack

Appétence
backend

Forces clés



Rigueur



Pragmatisme



Autodidacte



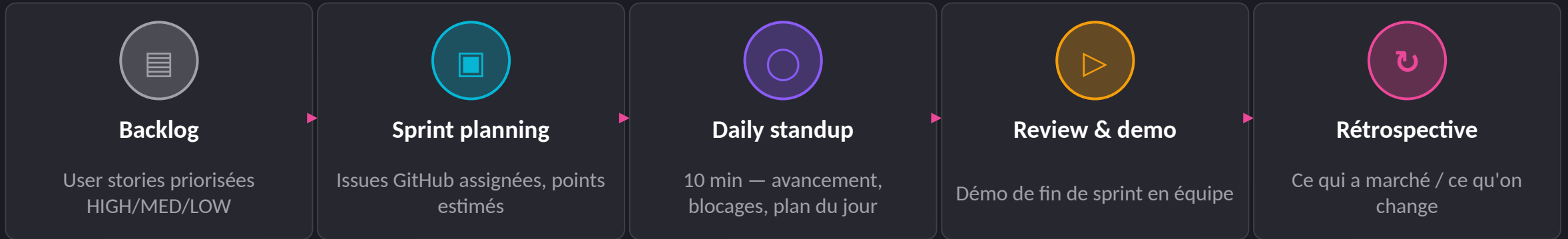
Métier ↔ Tech

20 ans de terrain, puis le code



Travail en mode agile

Cycle de développement adopté



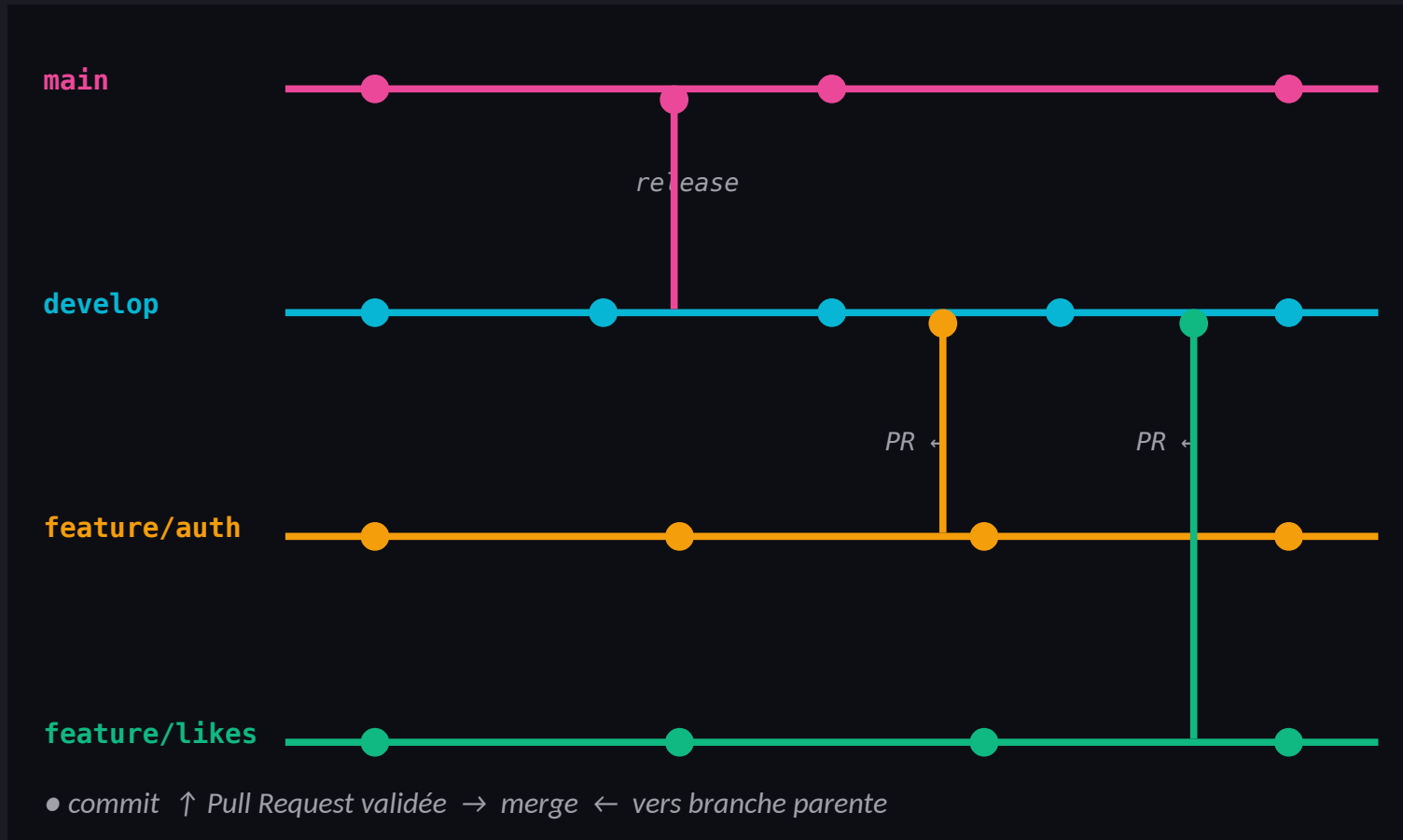
Outils & rituels

- **GitHub Issues**: Backlog vivant, labels, milestones
- **GitHub Projects (Kanban)**: Colonnes Todo / Doing / Review / Done
- **Pull Requests**: Code review obligatoire avant merge
- **GitHub Actions**: CI bloquante : lint + test + build

Découpe du projet

- 3** phases (MVP / Features / Polish)
- 5** sprints d'environ 10 jours
- 43** user stories réparties par sprint
- 1** review + rétro par sprint

Workflow Git en équipe



Règles d'équipe

- 1 feature = 1 branche
- PR obligatoire vers develop
- CI verte avant merge
- develop → main = release

6 contributeurs • 12+ issues majeures • PR #118 stabilisation Docker/Neon/Prisma/Redis

Anatomie d'une Pull Request

OPEN

#118 Stabilisation Docker/Neon/Prisma/Redis

arocchet wants to merge 18 commits into main ← from fix/docker-stabilisation

Commits (Conventional Commits)

- feat(docker): multi-stage build avec image Bun
- fix(prisma): openssl + migrate deploy au boot
- chore(deps): bump @upstash/redis to 1.35
- test(auth): add register integration test

Checks CI • GitHub Actions

- ✓ lint passed in 22s
- ✓ test passed in 1m 14s
- ✓ build passed in 2m 03s
- vercel preview deployed · pr-118.konekt.vercel.app

✓ Merge pull request 2 reviewers ✓ approved · all checks passed · no conflicts

Pratiques équipe



Conventional Commits



1 PR = 1 feature ou fix



CI verte = pré-requis



Review par ≥ 1 pair



Squash & merge

Konekt — contexte & vision

Problématique

Concevoir une application sociale moderne, sécurisée et temps réel, comme support de démonstration pour la certification CDA. Le projet doit illustrer l'ensemble du cycle : conception, développement, tests, déploiement.

Solution proposée

Konekt : posts, stories, reactions, messagerie privée et de groupe, événements, follow & friends, notifications temps réel — déployé en serverless avec une base relationnelle riche et un push SSE.



Réseautage

Posts, Stories, Reels, Reactions



Messagerie

DM, Group Chat, SSE temps réel



Communautés

Groupes, Événements, RSVP



Découverte

Search, Trending, Recommandations



Personnalisation

Thème, langue, confidentialité

User stories & périmètre

43

User stories

12+

Pages principales

18

Modèles Prisma

20+

Endpoints API

200+

Variables CSS

Top stories prioritaires

Acteur	Action	Priorité	Critère d'acceptation
Visiteur	S'inscrire	★★★	Email validé, compte créé, JWT émis
Utilisateur	Créer un post	★★★	Post visible immédiatement dans le feed
Utilisateur	Envoyer un message	★★★	Message persisté + push SSE au destinataire
Utilisateur	Réagir / commenter	★★	Compteur incrémenté, doublon impossible
Modérateur	Supprimer un contenu	★★	Contenu retiré, log d'action conservé

Architecture multicouche

PRÉSENTATION

UI / UX — ce que l'utilisateur voit

Pages Next.js (React Server Components)
Composants UI (shadcn/ui, Tailwind)
Hooks de données (use-api, use-real-time-chat)

APPLICATION / MÉTIER

Logique applicative et règles métier

Route Handlers /api/private/* /api/public/*
Server Actions (mutations) · Middleware JWT
Validation Zod, schémas partagés client/serveur

PERSISTANCE

Accès aux données

Prisma ORM (18 modèles, requêtes type-safe)
Migrations versionnées prisma/migrations
Helpers src/lib/server/<domaine>/<entité>.ts

INFRASTRUCTURE

Stockage et services externes

PostgreSQL Neon (DB relationnelle)
Upstash Redis (cache + buffer SSE)
Cloudinary (médias) · Vercel (runtime)

Stack technique

Frontend

Next.js 15

App Router • SSR/CSR hybride • Server Actions

React 19 + TypeScript

Composants typés, hooks personnalisés

Tailwind CSS + shadcn/ui

Design system, dark theme natif

Backend & API

Next.js Route Handlers

REST + Server-Sent Events

Prisma 6

ORM type-safe, migrations, 18 modèles

Zod

Validation des payloads, schémas partagés

Données & Temps réel

PostgreSQL (Neon)

Base relationnelle managée, branches

Upstash Redis (REST)

Cache + buffer SSE compatible serverless

Cloudinary

Stockage images / vidéos, transformations

Auth & Sécurité

JWT custom (jose)

Cookie HttpOnly • SameSite • Secret applicatif

bcrypt cost 12

Hash mots de passe, comparaison constante

OAuth Google

googleapis + state cookie anti-CSRF

DevOps & CI

Docker multi-stage

Image légère oven/bun:1

GitHub Actions

Lint • Test • Build • Deploy

Vercel + Neon

Déploiement préview par PR + prod

Qualité

Jest + ts-jest

Tests d'intégration API

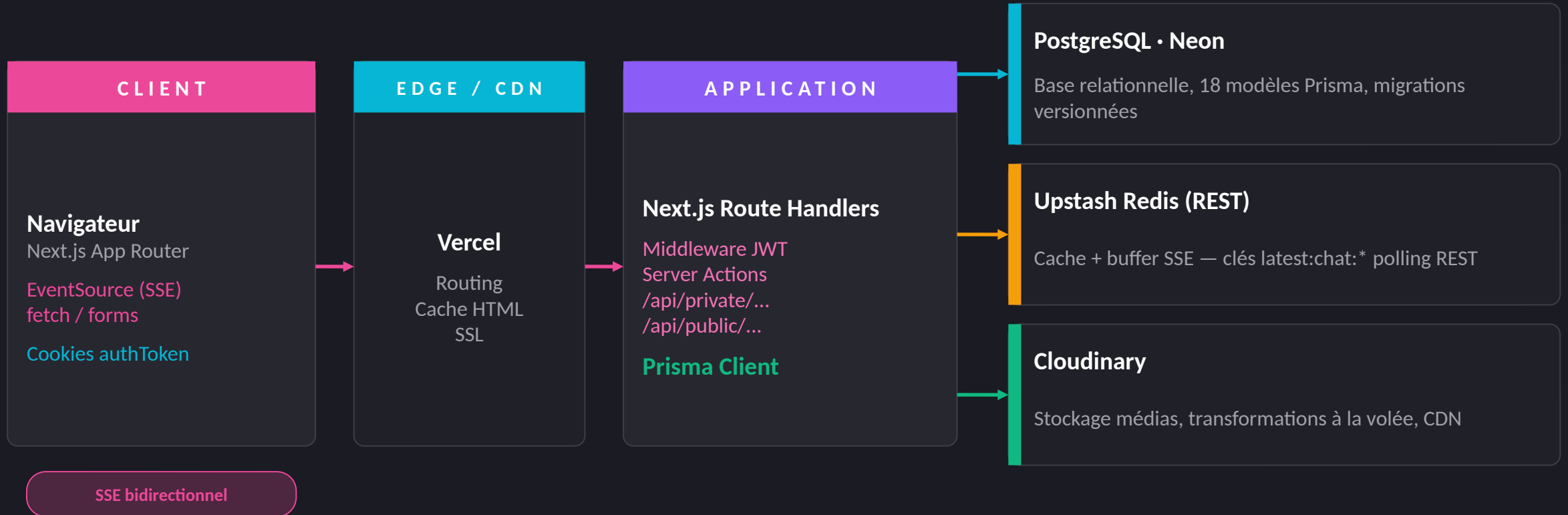
ESLint + TypeScript strict

Code style + typage

Roadmap Playwright

E2E parcours critiques

Architecture système



Tout est serverless-ready : la stack tourne nativement sur Vercel et reste reproductible en local via docker-compose.

Concepts de modélisation

Rappel des termes que l'on rencontrera dans MCD / MLD / MPD



Entité / Table

Objet métier matérialisé en table SQL. Une ligne = une instance.

Exemple Konekt

User, Post, Comment, Reaction



Attribut / Colonne

Caractéristique d'une entité, typée. Devient une colonne en base.

Exemple Konekt

Useremail (varchar) · Postmessage (text)



Clé primaire (PK)

Identifiant unique non-null d'une ligne. Une seule par table.

Exemple Konekt

User.id (uuid) · Post.id (cuid)



Clé étrangère (FK)

Référence la PK d'une autre table. Maintient l'intégrité référentielle.

Exemple Konekt

Post.userId → User.id



Cardinalité

Nombre d'instances liées : 1-1, 1-N, N-N. Détermine la structure relationnelle.

Exemple Konekt

User 1– N Post · User N– N User via Friendship



Index

Structure d'accès accélérant les lectures sur certaines colonnes.

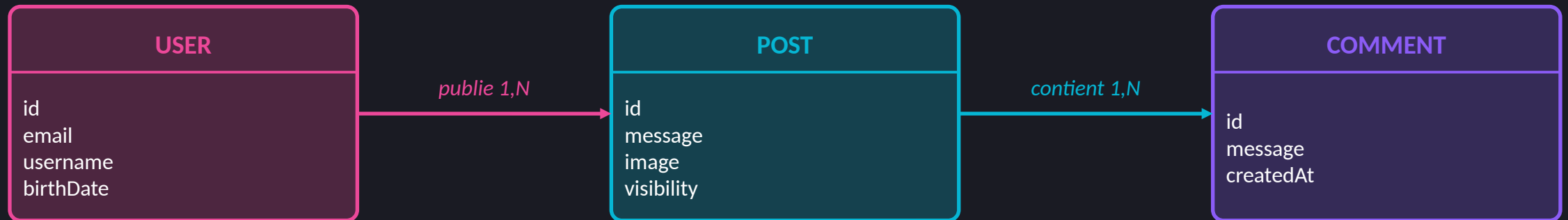
Exemple Konekt

@ @ index([userId, createdAt]) sur Post

Modèle Conceptuel de Données

Définition

Représentation abstraite des entités métier et de leurs relations, indépendante de toute technologie. On y identifie les entités, leurs attributs et les cardinalités entre elles. C'est la vue "business" du domaine.



Lecture du MCD

- Un **User** publie 0..N **Post** — un Post a exactement 1 auteur (cardinalité 1,1 côté Post)
- Un **Post** contient 0..N **Comment**, chaque Comment référence exactement 1 Post (parent)
- Le MCD n'expose *ni clés étrangères, ni types techniques* — uniquement le sens métier

Modèle Logique de Données

Définition

Traduction du MCD en schéma relationnel : on matérialise les relations par des clés étrangères, on choisit les types logiques (string, int, datetime) et on précise les contraintes (NOT NULL, UNIQUE).

●●● MLD – extrait (notation Prisma)

USER (id PK, email UNIQUE, username UNIQUE, password, birthDate, createdAt)

POST (id PK, userId FK → USER.id, message, inAge?, visibility, createdAt)

COMMENT (id PK, postId FK → POST.id, userId FK → USER.id, message, createdAt)

REACTION (id PK, userId FK → USER.id, postId FK? → POST.id, type, createdAt)

UNIQUE (userId, postId, type) ← un utilisateur ne peut pas dupliquer une réaction

STORY (id PK, userId FK → USER.id, inAge, expiresAt, createdAt)

FRIENDSHIP (id PK, requesterId FK → USER.id, addresseeId FK → USER.id, status)

UNIQUE (requesterId, addresseeId)

À retenir : chaque relation N-1 du MCD devient une **FK** côté "plusieurs". Les relations N-N (ex. Friendship) deviennent une table d'association avec ses propres attributs.

Modèle Physique de Données

Définition

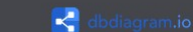
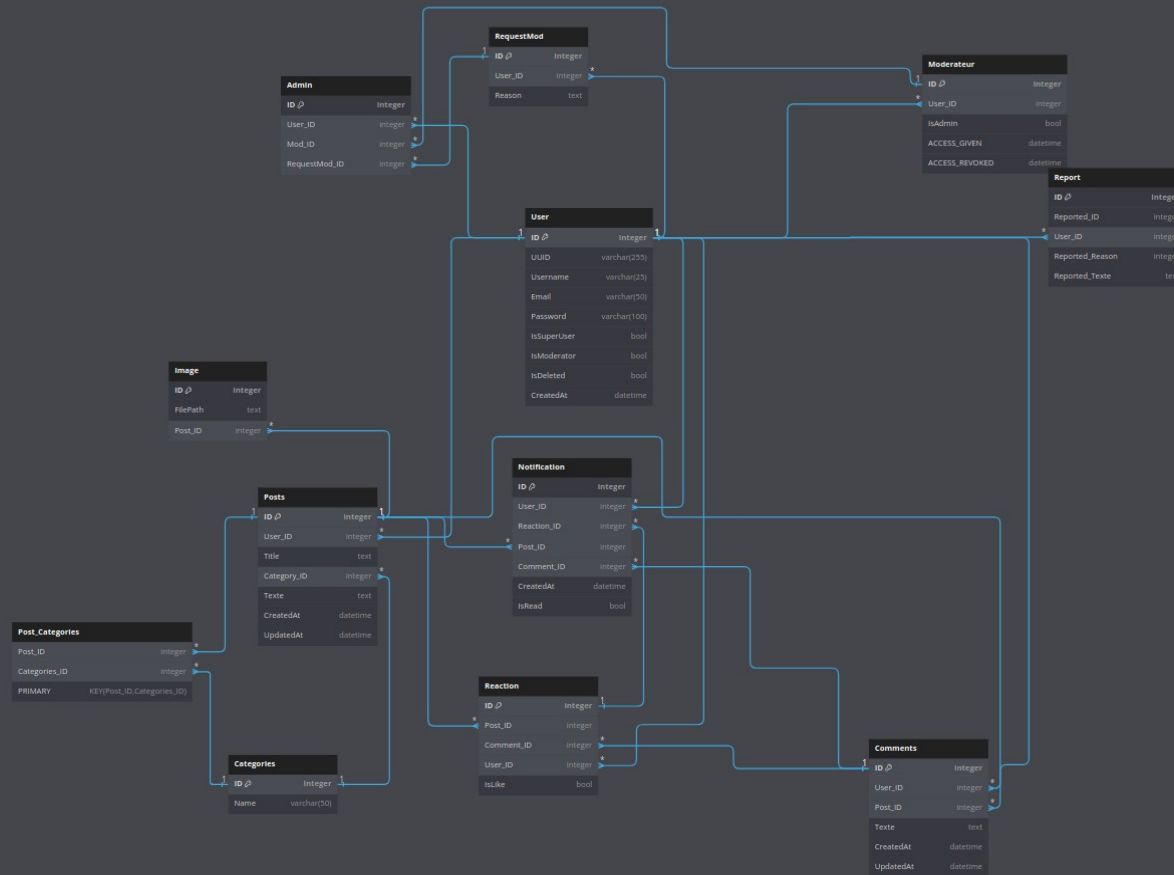
Implémentation SQL concrète du MLD pour un SGBD donné — ici PostgreSQL. On y choisit les types techniques (varchar(n), uuid, timestampz), les contraintes (CHECK, ON DELETE), les index physiques.

● ● ● MPD – extrait DDL PostgreSQL

```
CREATE TABLE "User" (
  id          TEXT          PRIMARY KEY DEFAULT cuid(),
  email       VARCHAR(255) NOT NULL UNIQUE,
  username    VARCHAR(50)  NOT NULL UNIQUE,
  password    VARCHAR(255) NOT NULL,
  visibility  "ProfileVisibility" NOT NULL DEFAULT 'PUBLIC',
  createdAt   TIMESTAMPTZ  NOT NULL DEFAULT NOW ()
);

CREATE TABLE "Post" (
  id          TEXT          PRIMARY KEY DEFAULT cuid(),
  userId      TEXT          NOT NULL REFERENCES "User"(id) ON DELETE CASCADE,
  message     TEXT          NOT NULL CHECK (length(message) <= 5000),
  visibility  "Visibility"  NOT NULL DEFAULT 'PUBLIC',
  createdAt   TIMESTAMPTZ  NOT NULL DEFAULT NOW ()
);
CREATE INDEX idx_post_user_created ON "Post"(userId, createdAt DESC);
```


Modèle de données — 18 entités



Faits saillants

18 Modèles Prisma

40+ Relations

7 Enums métier

35+ Attributs User

Entités centrales

- User
- Post / Comment / Reaction
- Story
- Conversation / Message
- Group / Event / Rsvp
- Notification

User au cœur du domaine

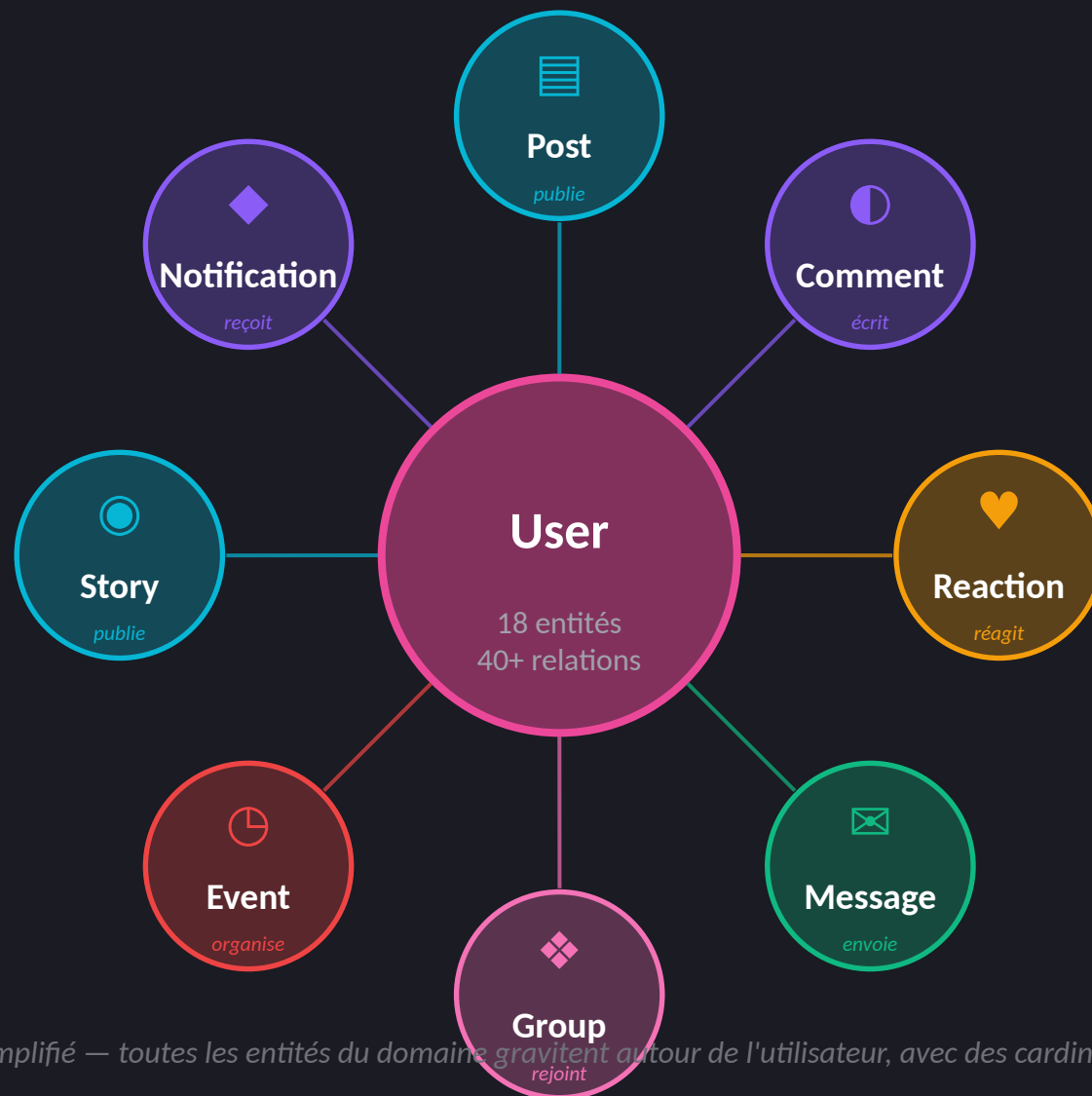
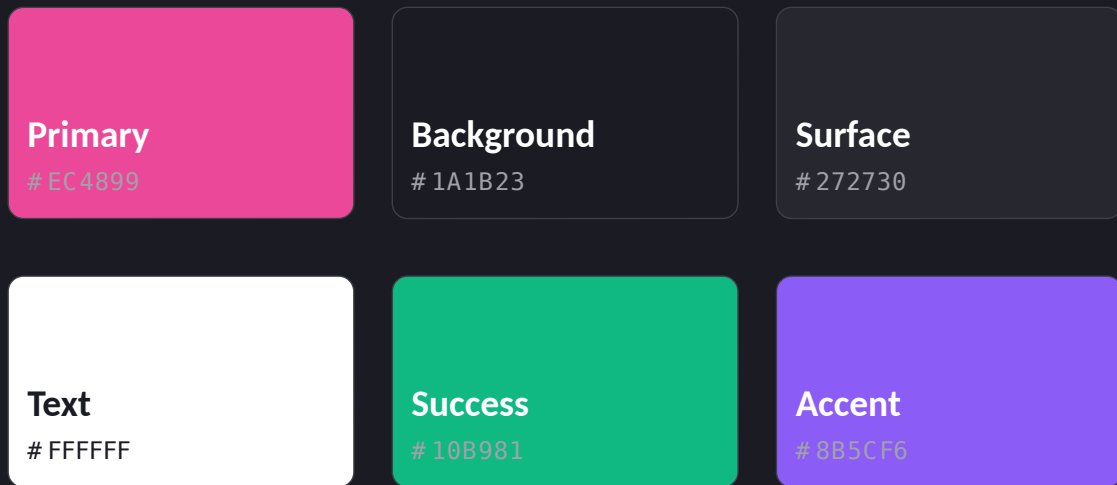


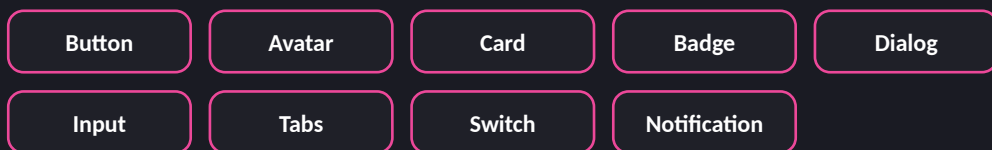
Diagramme conceptuel simplifié — toutes les entités du domaine gravitent autour de l'utilisateur, avec des cardinalités explicites côté Prisma.

Design system Konekt

Palette



Composants



Typographie

Aa

Geist

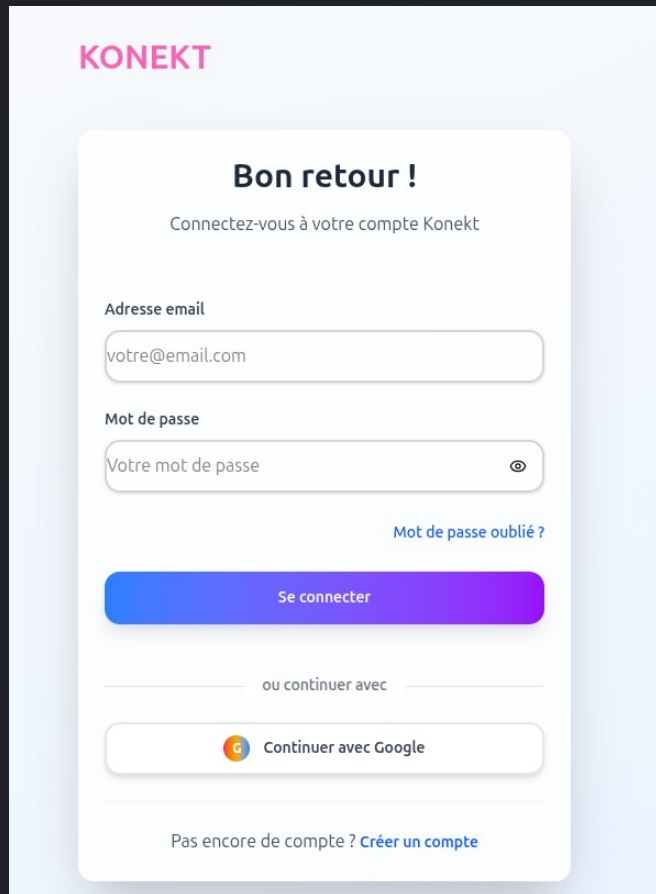
Font family — Next.js next/font · variable, optimisée

H1	32 px
H2	24 px
Body	16 px
Small	12 px

Breakpoints

☐ Mobile	☒ Tablet	☒ Desktop
< 640px	640 – 1024px	> 1024px

Maquette — Login



Maquette de la page de login Konekt. Le design est moderne et épuré, avec une palette de couleurs dominée par le blanc et le gris, ponctuée de touches de bleu et de rose. Le logo KONEKT est en rose en haut à gauche. Le titre principal est 'Bon retour !' en gras, suivi de l'instruction 'Connectez-vous à votre compte Konekt'. Les champs de saisie pour l'adresse email et le mot de passe sont blancs avec une bordure grise. Le mot de passe est masqué par des points et dispose d'un bouton 'oeil' pour basculer la visibilité. Un lien 'Mot de passe oublié ?' est en bleu. Le bouton 'Se connecter' est large, avec un dégradé de bleu à rose. En dessous, une section 'ou continuer avec' est séparée par une ligne pointillée. Elle contient un bouton 'Continuer avec Google' avec le logo Google. En bas, un lien 'Créer un compte' est en bleu.

KONEKT

Bon retour !
Connectez-vous à votre compte Konekt

Adresse email
votre@email.com

Mot de passe
Votre mot de passe

[Mot de passe oublié ?](#)

Se connecter

ou continuer avec

 Continuer avec Google

Pas encore de compte ? [Créer un compte](#)

Login

AUTHENTIFICATION

Page d'entrée Konekt — design soigné, validation côté client par Zod, lien vers OAuth Google. Pas de fuite d'information sur l'existence d'un compte côté serveur.

Points techniques clés

JWT signé

jose + secret applicatif

Cookie HttpOnly

SameSite=Lax · pas d'accès JS

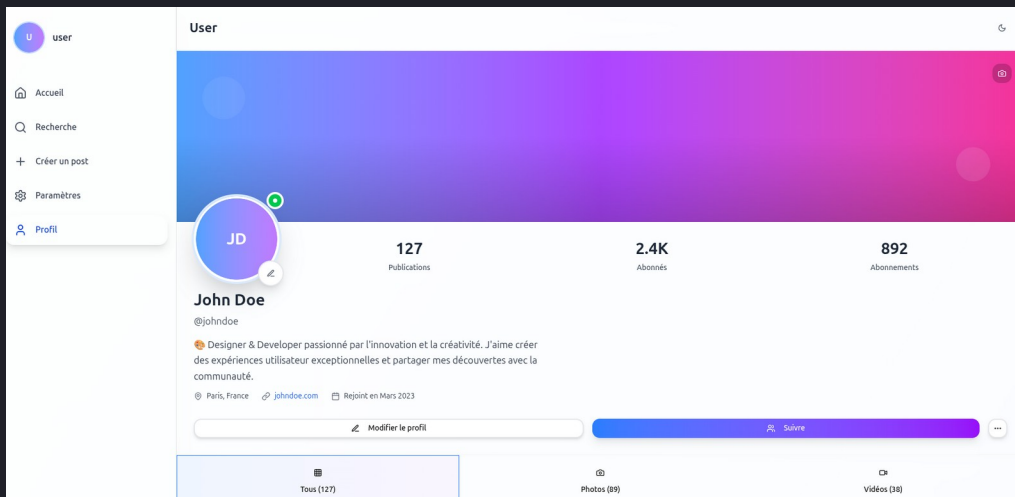
OAuth Google

state cookie anti-CSRF

Validation Zod

schéma partagé client / serveur

Maquette — Home Feed



Home Feed

VUE PRINCIPALE

Le feed est le cœur de Konekt : posts amis, stories en haut, suggestions de profils, infinite scroll, image full-width. Tout est servi par `/api/private/post` avec pagination cursor-based.

Points techniques clés

Infinite scroll

Pagination cursor-based

Stories ephemeral

24h, polling SSE pour le statut

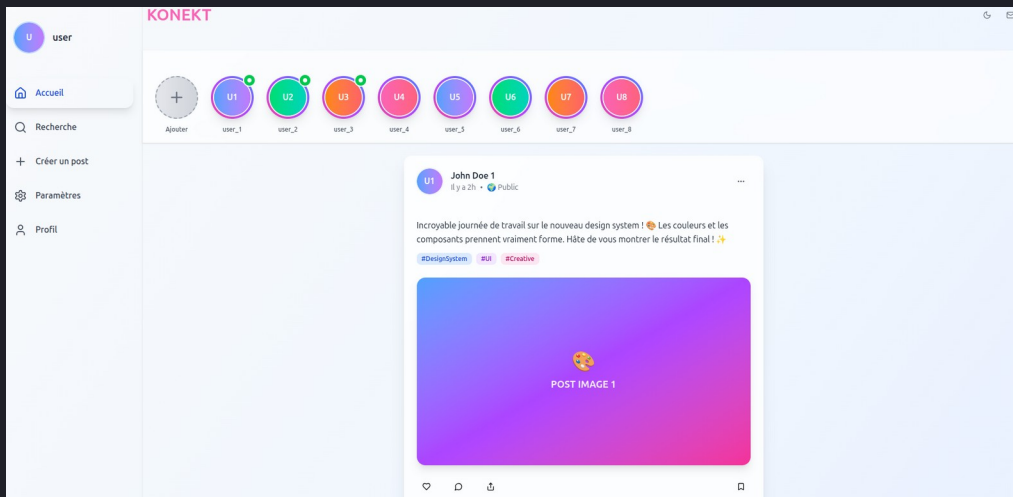
Reactions inline

Optimistic UI · doublon impossible

Image optimisée

Next/Image + Cloudinary

Maquette — Profil utilisateur



Profil utilisateur

IDENTITÉ

Profil public ou privé selon ProfileVisibility. Compteurs followers / following / posts, gradient personnalisable, tabs Posts / Photos / Vidéos, action Suivre / Modifier.

Points techniques clés

Visibilité PUBLIC/PRIVATE

Enum Prisma respecté côté API

Followers / Following

Modèle Friendship 2-way

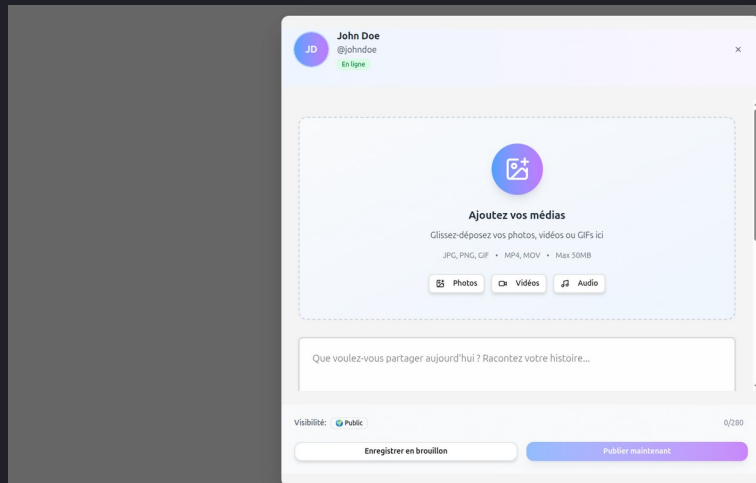
Tabs dynamiques

Posts / Photos / Vidéos

Edit profile

Server Action + Cloudinary upload

Maquette — Création de post



Création de post

PUBLICATION

Composer riche avec preview, upload Cloudinary, choix de visibilité (PUBLIC / FRIENDS / PRIVATE), validation Zod côté serveur, persistance Prisma puis broadcast SSE.

Points techniques clés

Upload Cloudinary

Direct upload signé côté serveur

Visibilité granulaire

3 niveaux + check au fetch

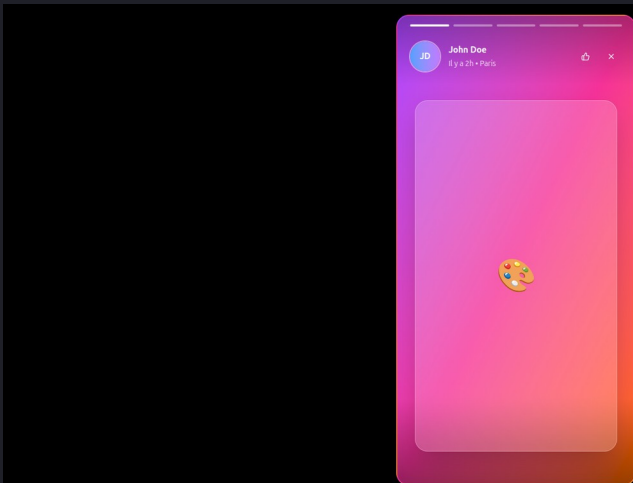
Validation Zod

Limite taille, type MIME, longueur

Server Action

Mutation type-safe Next.js

Maquette — Story Viewer



Story Viewer

CONTENU ÉPHÉMÈRE

Lecteur plein écran des stories des amis, progression segmentée, swipe navigation, expiration automatique à 24h. Statistique de vues persistée pour l'auteur.

Points techniques clés

Stories 24h

Job cron • cleanup expirés

Progress segmentée

Multi-stories par utilisateur

Vues uniques

Compteur stocké en relation

Reactions emoji

Quick reply au créateur

Maquette — Paramètres

Paramètres

PERSONNALISATION

Page settings centralisée — profil, sécurité (mot de passe, sessions), notifications, langue (FR/EN), thème, confidentialité. Préférences stockées dans UserSettings (1-to-1 User).

Points techniques clés

UserSettings

Modèle Prisma 1-to-1 dédié

Theme toggle

Stocké côté serveur · cohérent

i18n EN / FR

next-intl, dictionnaires lazy

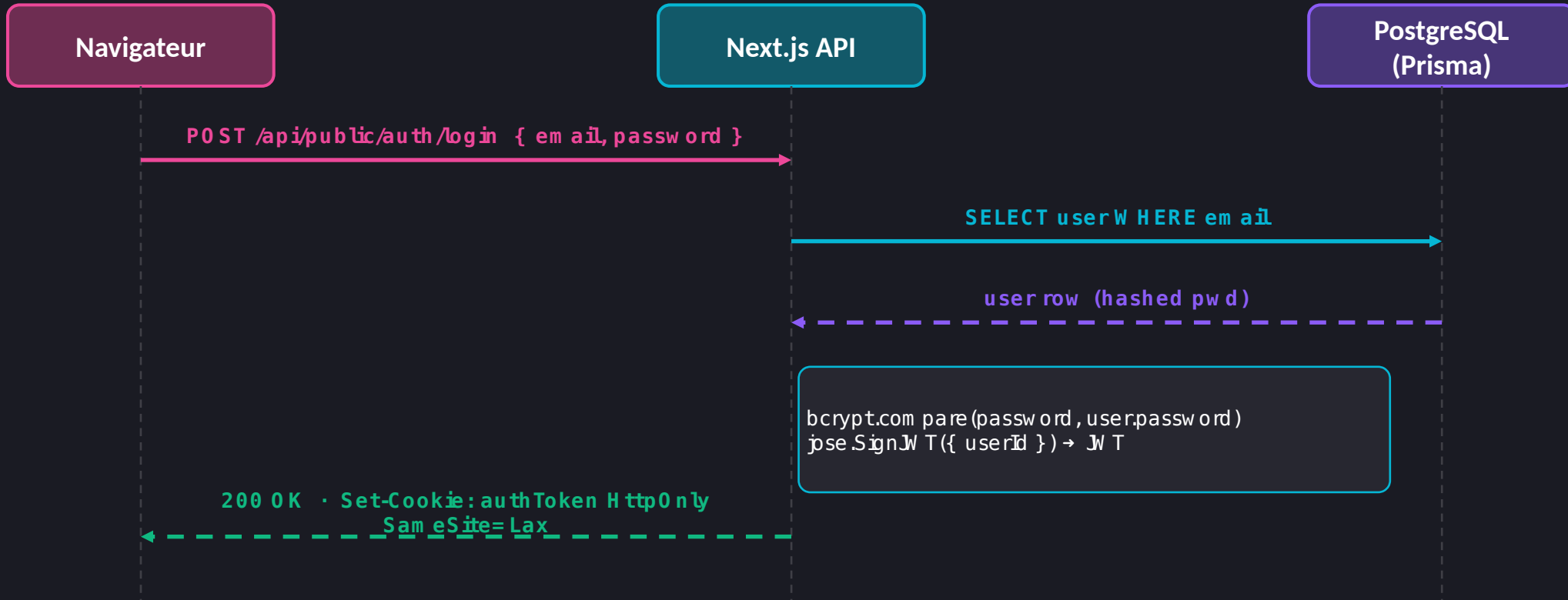
Privacy controls

ProfileVisibility, notification opt-in

Pages implémentées • 14 routes

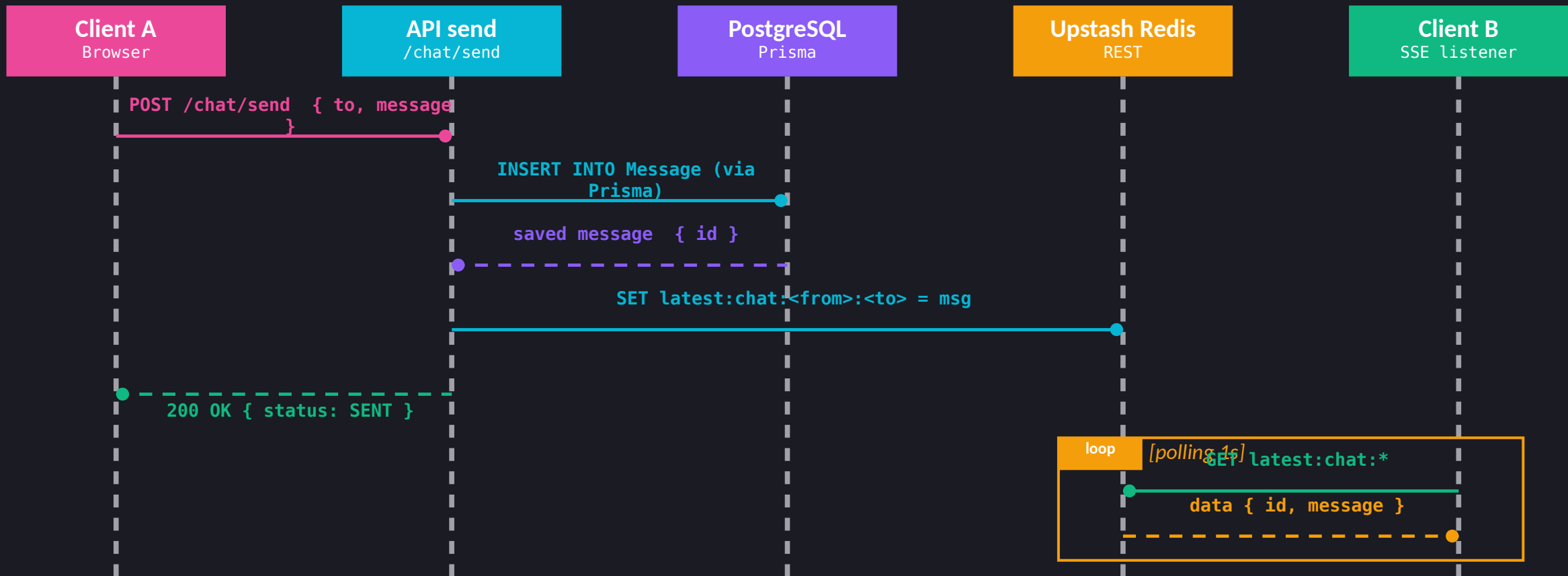
● /	Feed	Affichage des posts du réseau	● /login	Login	Authentification email + OAuth Google
● /register	Register	Inscription avec vérification	● /onboarding	Onboarding	Complétion du profil
● /profile/[id]	Profil	Profil public/privé, posts, abonnés	● /chat	Conversations	Liste des messages 1-to-1
● /chat/[id]	Chat DM	Messagerie privée temps réel (SSE)	● /reels	Reels	Vidéos courtes verticales
● /search	Recherche	Users + posts avec historique	● /groups	Groupes	Liste, recherche, création
● /groups/[id]	Groupe	Feed + chat + membres	● /events	Événements	Calendrier + RSVP
● /settings	Paramètres	Profil, sécurité, langue, thème	● /invitations	Invitations	Demandes d'amitié et de groupe

Flux d'authentification



Sécurité — JWT signé avec **jose** + secret applicatif · Cookie **HttpOnly** (pas accessible JS) · **bcrypt** cost 12 · **OAuth Google** via state cookie anti-CSRF · Middleware Next.js qui valide le JWT pour chaque route privée.

Temps réel — envoi message en SSE



Auth — signature JWT (jose)

src/lib/server/user/login.ts

```
import { SignJWT } from 'jose';
import bcrypt from 'bcrypt';

export async function login(email: string, password: string) {
  const user = await db.user.findUnique({ where: { email } });
  if (!user) throw new AuthError('INVALID_CREDENTIALS');

  const ok = await bcrypt.compare(password, user.password);
  if (!ok) throw new AuthError('INVALID_CREDENTIALS');

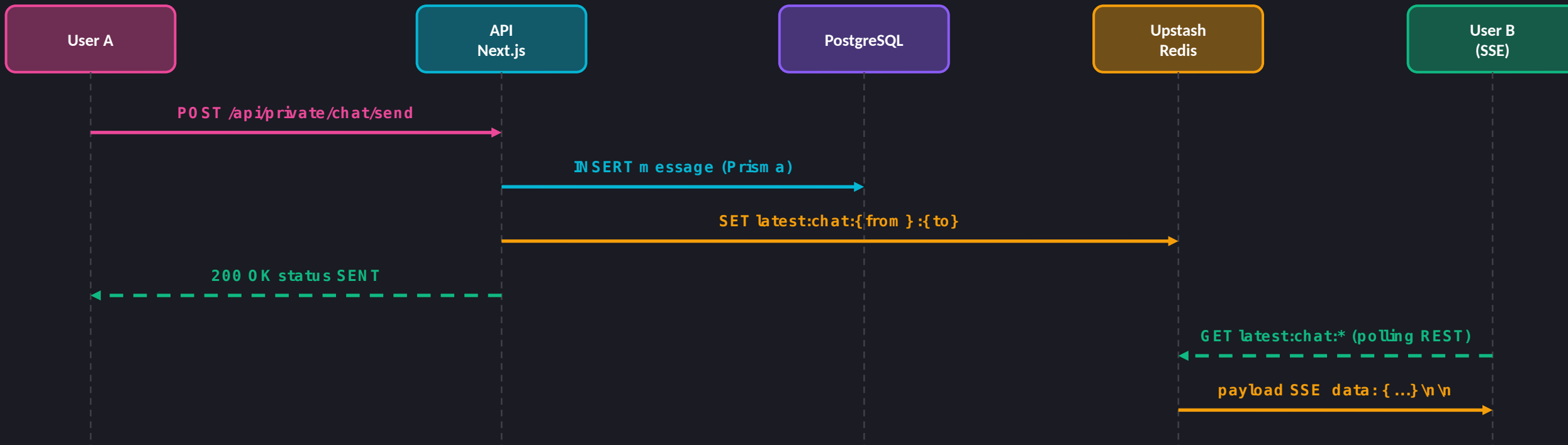
  const secret = new TextEncoder().encode(process.env.JWT_SECRET);
  const token = await new SignJWT({ userId: user.id })
    .setProtectedHeader({ alg: 'HS256' })
    .setIssuedAt()
    .setExpirationTime('7d')
    .sign(secret);

  return { token, user };
}
```

Points clés

- ✓ bcrypt.compare en temps constant
- ⦿ Même message si user inconnu
- ⊕ JWT signé HS256 (jose)
- 🕒 Expiration explicite 7 jours
- ✦ Secret env var jamais commité
- ❏ Erreurs typées (AuthError)

Temps réel — Server-Sent Events



Compromis assumé · Upstash Redis est un service REST (pas de pub/sub TCP) : on poll la clé `latest:chat:*` depuis l'endpoint SSE, ce qui reste compatible avec Vercel serverless et suffit pour le volume du projet. Roadmap : Redis classique en pub/sub si la charge augmente.

SSE — endpoint /chat/listen

●●● src/app/api/private/chat/listen/route.ts

```
export async function GET(req: NextRequest) {
  const userId = await getUserIdFromRequest(req);
  if (!userId) return new Response('Unauthorized', { status: 401 });

  const { conversationId, type } = parseParams(req.url);

  const stream = new ReadableStream({
    start(controller) {
      const enc = new TextEncoder();
      controller.enqueue(enc.encode(`data: {"type":"connected"}\n\n`));

      let active = true, lastId = '';
      (async function poll() {
        while (active) {
          const key = `latest:chat:${userId}:${conversationId}`;
          const data = await redis.get(key);
          if (data && data.id !== lastId) {
            lastId = data.id;
            controller.enqueue(enc.encode(`data: ${JSON.stringify(data)}\n\n`));
          }
          await sleep(1000);
        }
      })();
      req.signal.addEventListener('abort', () => { active = false; controller.close(); });
    },
  });
  return new Response(stream, { headers: { 'Content-Type': 'text/event-stream' } });
}
```

Mécanique



ReadableStream Web standard



Polling Redis 1s (REST)



Clé latest:chat:<from>:<to>



Déduplication par lastId



abort → cleanup poll



userId injecté par middleware

API • 60+ endpoints REST

Auth

POST /api/public/auth/register

POST /api/public/auth/login

POST /api/public/auth/logout

GET /api/public/auth/callback

User & Profil

GET /api/private/me

PUT /api/private/me

GET /api/private/friend-requests

POST /api/private/friend-requests

Contenu

GET /api/private/post

POST /api/private/post

GET /api/private/stories

POST /api/private/stories

Messagerie & Temps réel

GET /api/private/conversations

POST /api/private/chat/send

SSE /api/private/chat/listen

SSE /api/private/chat/typing/listen

Communautés

GET /api/private/groups

POST /api/private/groups

GET /api/private/events

POST /api/private/events

Recherche & Notif

GET /api/private/search

GET /api/private/invitations

GET /api/private/notifications

PATCH /api/private/notifications/:id/read

Middleware d'authentification

src/middleware.ts

```
export async function middleware(req: NextRequest) {
  const { pathname } = req.nextUrl;
  const isPublic = pathname.startsWith('/api/public')
    || ['/login', '/register'].some(r => pathname.startsWith(r));
  if (isPublic) return NextResponse.next();

  const token = req.cookies.get('authToken')?.value;
  if (!token) return NextResponse.redirect('/login');

  try {
    const { userId } = await verifyJwt(token);
    const headers = new Headers(req.headers);
    headers.set('x-user-id', userId);
    return NextResponse.next({ request: { headers } });
  } catch {
    return NextResponse.redirect('/login');
  }
}
```

Points clés

-  Allow-list pour routes publiques
-  Cookie HttpOnly authToken
-  Vérification JWT via jose
-  Redirect /login si invalide
-  Injection x-user-id header
-  Sécurité par défaut

Sécurité applicative & RGPD

Mesures en place

- JWT signé (jose) · cookies HttpOnly · SameSite=Lax
- ✦ bcrypt cost 12 sur tous les mots de passe
- ⊕ Validation Zod systématique des payloads
- ▣ OAuth Google avec state cookie anti-CSRF
- ⊖ SQL injection bloquée par Prisma (paramétrage)
- 🔍 OWASP Top 10 audité — voir securite-rgpd.md
- HTTPS forcé en production via Vercel

Conformité RGPD

Données collectées

Email, nom, date de naissance, contenus publiés. Aucun traitement marketing ; aucune donnée sensible au sens art. 9.

Droits de la personne

Accès & consultation profil



Rectification (settings)



Effacement / droit à l'oubli

Roadmap

Portabilité (export JSON)

Roadmap

Opposition / consentement



Roadmap durcissement : rate-limit, headers CSP, sanitization markdown (DOMPurify).

Cheminement d'une requête SQL

```
●●● src/app/api/private/post/[id]/route.ts

import { db } from '@lib/db';
import { getUserIdFromRequest } from '@lib/server/api/getUserId';
export async function GET(req: NextRequest, ctx) {
  const userId = await getUserIdFromRequest(req);
  if (!userId) return respondError('Unauthorized', 401);
  const { id } = await ctx.params;
  // 1. Validation entrée – Zod
  const params = z.object({ id: z.string().cuid() }).parse({ id });
  // 2. Query Prisma type-safe → SQL paramétré
  const post = await db.post.findUnique({
    where: { id: params.id },
    include: {
      user: { select: { id, username, avatar } },
      comments: { select: { id, message, datetime } },
      reactions: true,
    },
  });
  // 3. Retour structuré + types inférés
  return respondSuccess(post);
}
```

Pipeline en 5 étapes

- Middleware → x-user-id
- Zod valide l'entrée
- Prisma compose le SQL
- pg envoie SQL paramétré
- Réponse typée TypeScript

Requête préparée : pourquoi c'est sûr

❓ Requête naïve — VULNÉRABLE

```
// ⚠ N'EST PAS dans notre code, illustration

const email = req.body.email;

const result = await db.$queryRawUnsafe(
  `SELECT * FROM users
   WHERE email = '${email}'`
);

// Si email = "x' OR '1'='1"
// → SELECT * FROM users
//   WHERE email = 'x' OR '1'='1'
// → renvoie TOUS les utilisateurs ☹
```

✓ Prisma — PROTÉGÉ par défaut

```
// ☐ Le pattern utilisé partout dans le projet

const email = req.body.email;

const user = await db.user.findUnique({
  where: { email },
});

// Prisma envoie au driver pg :
//   SELECT * FROM users WHERE email = $1
//   avec params = ['x' OR '1'='1']
//
// PostgreSQL traite $1 comme STRING littérale
// → 0 ligne renvoyée ✓
```

Anti-SQLi : défense en profondeur

1

ZOD

Validation type + format avant tout

```
z.string().email().min(6)
```

2

PRISMA

Compose un SQL paramétré (jamais de concat)

```
db.user.findUnique({ where: { email } })
```

3

DRIVER PG

Sépare query texte + array de params

```
PREPARE → EXECUTE($1, $2, ...)
```

4

POSTGRES

Traite chaque param comme littéral

```
WHERE email = $1 (binding)
```

Payload attaque



' OR 1=1-- envoyé



Zod : pas un email



Prisma : paramètre



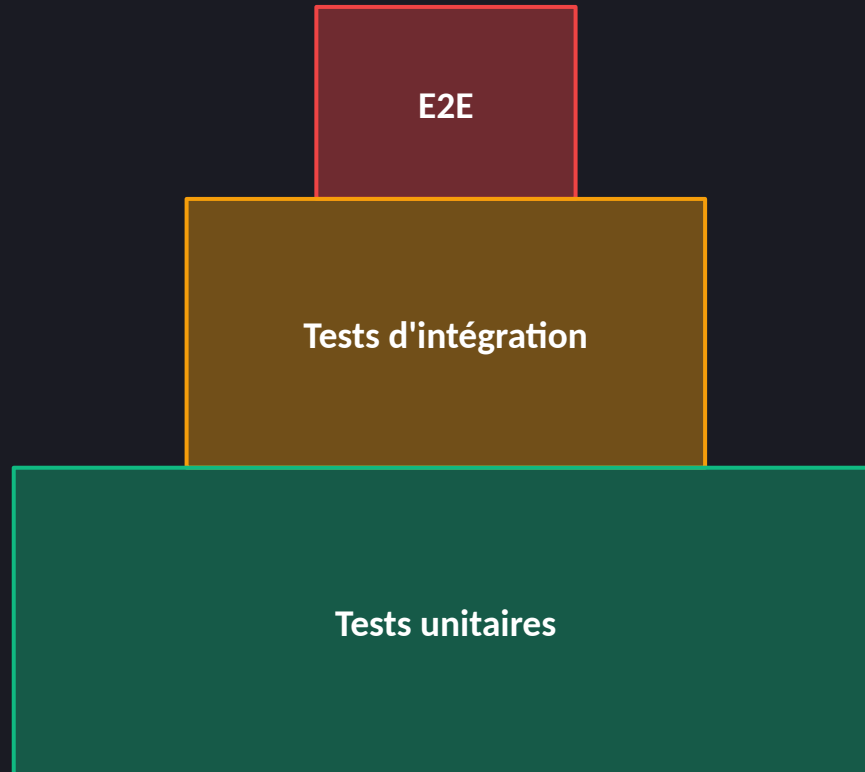
Driver : binding



PG : 0 résultat

OWASP Top 10 — A03:2021 Injection • neutralisée structurellement par Prisma

Stratégie de tests



Pyramide adoptée

E2E (Playwright)

[Roadmap](#)

Parcours critiques : inscription, post, chat

Intégration API

[En place](#)

Jest + ts-jest sur /api/* (auth, posts, chat)

Unitaires

[En place](#)

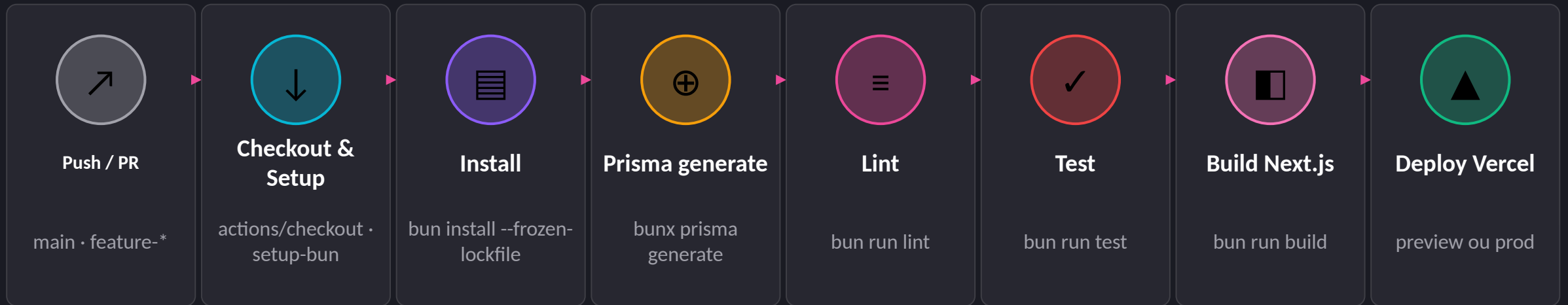
Helpers, validators Zod, hashing, JWT

Lint & types

[CI bloquant](#)

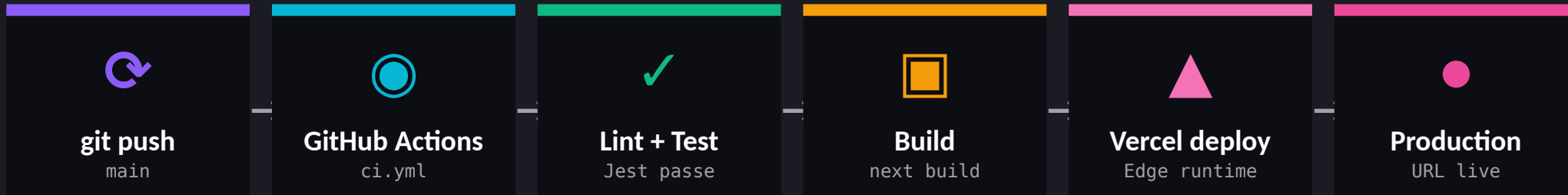
Coverage cible : 80% sur les modules critiques (auth, chat, persistence). Jeux d'essai documentés dans tests-strategy.md.

Pipeline CI/CD GitHub Actions



```
.github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps: [checkout, setup-bun, install, prisma generate, lint, test, build]
```

Du commit à la production



●●● .github/workflows/ci.yml – déclencheurs

name: CI

on:

push:

branches: [main] # auto-deploy sur main

pull_request: # check sur PR

branches: [main, develop]

workflow_dispatch: # bouton manuel

Chiffres clés

● ~2 min commit → preview

● Lint + Test bloquants

● Vercel preview par PR

Workflow GitHub Actions — détail

```
●●● .github/workflows/ci.yml

name: CI
on:
  push:      { branches: [main] }
  pull_request:
jobs:
  build:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
      - uses: oven-sh/setup-bun@v1
        with: { bun-version: '1' }
      - name: Install deps
        run: bun install --frozen-lockfile
      - name: Prisma generate
        run: bunx prisma generate
      - name: Lint
        run: bun run lint
      - name: Tests d'intégration
        run: bun run test
      - name: Build Next.js
        run: bun run build
```

Que fait chaque étape ?

- checkout : clone le repo
- setup-bun : runtime CI
- bun install : deps lockées
- prisma generate : types
- lint + test : qualité
- build : artefact .next

Triggers : 3 façons de déclencher



push sur main

Merge d'une PR sur main

CI complet + auto-deploy production

```
on:
  push:
    branches:
      - main
```



Déploiement auto
en production



pull_request

Ouverture / update d'une PR

CI + Preview Vercel + Neon branch isolée

```
on:
  pull_request:
    branches: [main, develop]
    types: [opened, synchronize]
```



Status check obligatoire
avant merge



workflow_dispatch

Bouton manuel dans GitHub UI

Rejeu d'un déploiement, debug

```
on:
  workflow_dispatch:
    inputs:
      env: { type: choice }
```



Lancement manuel
pour rebuild d'urgence

Dockerfile — image applicative

● ● ● Dockerfile

```
# Stage 1 — deps (cache layer)
FROM oven/bun:1 AS deps
WORKDIR /app
COPY package.json bun.lock ./
RUN bun install --frozen-lockfile

# Stage 2 — builder
FROM oven/bun:1 AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN bunx prisma generate && bun run build

# Stage 3 — runner (image légère)
FROM oven/bun:1-slim AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/.next ./next
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./
EXPOSE 3000
CMD ["bun", "run", "start"]
```

Choix techniques

-  Multi-stage : cache deps séparé
-  Bun runtime (build & run)
-  prisma generate avant build
-  Image slim finale (~150 Mo)
-  Pas de secret dans l'image
-  Port 3000 explicite

docker-compose.yml — env local

● ● ● docker-compose.yml

```
services:
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgres://postgres:postgres@db:5432/social
      REDIS_URL: Upstash Redis REST (UPSTASH_REDIS_REST_URL)
      JWT_SECRET: ${JWT_SECRET}
    depends_on: [db, redis]

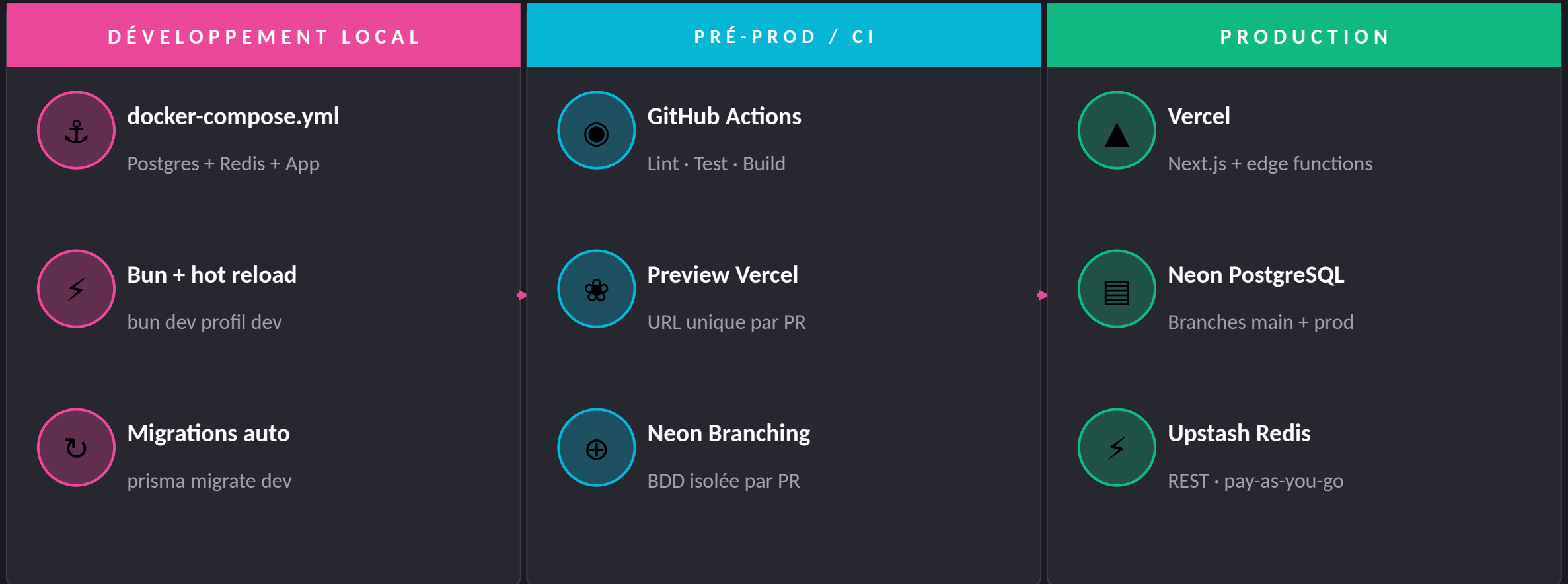
  db:
    image: postgres:16-alpine
    ports: ["5432:5432"]
    environment:
      POSTGRES_DB: social-network
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
```

Pourquoi compose ?

-  Reproduit prod en local
-  DB + Redis + App orchestrés
-  depends_on respecté
-  Variables d'env via .env
-  docker compose up = ready
-  Profil dev (hot reload)

Production • Vercel + Neon + Upstash



Rollback Vercel one-click • Healthcheck applicatif • Secrets via env vars (jamais committés) • SSL automatique.

Veille technique

Sources suivies



Vercel Blog

Annonces Next.js, App Router



React Docs (canary)

Server Components, Actions



OWASP Cheat Sheets

Sécurité applicative



RFC / WHATWG specs

HTTP, cookies, fetch



Twitter dev community

Theo, Lee Robinson, Dan Abramov



YouTube — Theo, Fireship

Synthèses & démos

Méthode

1

Filtrer

Critère : signal-to-noise élevé, sources primaires d'abord

2

Lire

30 min / matin, Pocket pour le offline

3

Tester

Petite preuve de concept si la techno entre dans le projet

4

Documenter

Notes dans Obsidian, lien vers la source

5

Partager

Discussion équipe sur les choix structurants

Métriques du projet

18

modèles
Prisma

60+

endpoints
API

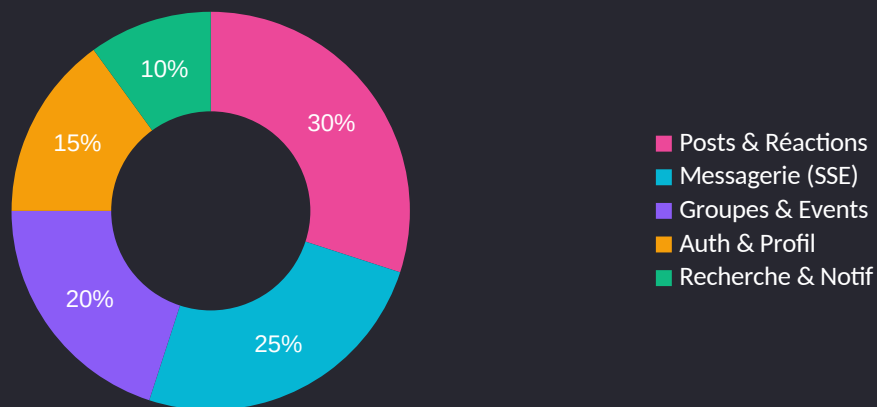
12+

pages
principales

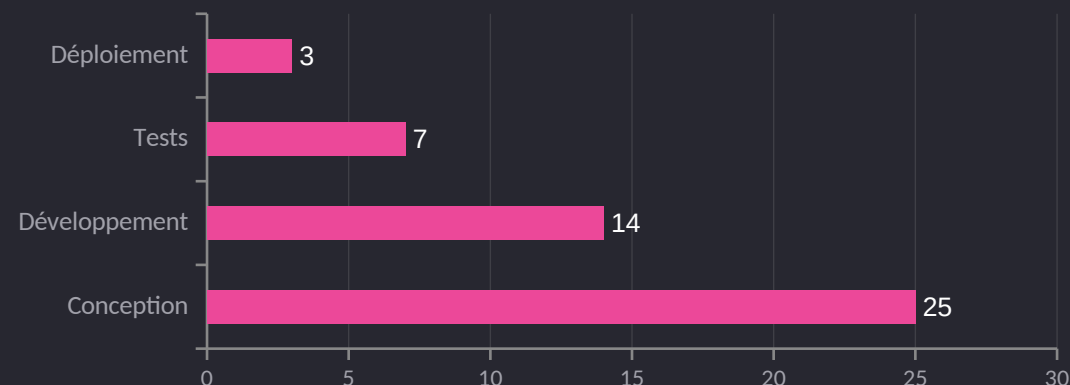
43

user
stories

Répartition des features livrées



Charge de travail par phase (jours)



Développer une application sécurisée

BC01

Bloc de compétences

RNCP 37873 • CDA

En quoi le bloc consiste

Concevoir, développer et tester une application en intégrant les principes de sécurité dès la phase de conception.

Compétences couvertes par Konekt

Maquetter une application

Wireframes + mockups 6 pages principales, design system Konekt dédié

Développer une interface utilisateur

Next.js App Router, React Server Components, Tailwind + shadcn/ui

Développer des composants réutilisables

Button, Avatar, Card, Dialog, Input — bibliothèque cohérente

Intégrer la sécurité

JWT signé (jose), bcrypt cost 12, cookies HttpOnly, validation Zod, OWASP Top 10

Preuves du dossier

Sécurité applicative

Validation Zod

Middleware JWT

Audit RGPD



Concevoir et développer la persistance des données

BC02

Bloc de compétences

RNCP 37873 • CDA

En quoi le bloc consiste

Modéliser une base de données relationnelle, écrire les requêtes optimisées et garantir l'intégrité.

Compétences couvertes par Konekt

Analyser le besoin et concevoir un MCD

43 user stories → 18 entités identifiées, cardinalités explicites

Mettre en place une base relationnelle

PostgreSQL Neon, migrations versionnées prisma/migrations

Développer des composants d'accès aux données

Prisma ORM type-safe, helpers src/lib/server/<domaine>/<entité>.ts

Garantir l'intégrité

PK, FK, UNIQUE composées (réactions), ENUM métier, cascades maîtrisées

Preuves du dossier

MCD / MLD / MPD

18 modèles Prisma

40+ relations

7 enums



Concevoir et développer une application multicouche

BC03

Bloc de compétences
RNCP 37873 • CDA

En quoi le bloc consiste

Construire une application répartie front + back avec une logique métier propre, des API et du temps réel.

Compétences couvertes par Konekt

Architecture multicouche

Présentation → Métier → Persistance → Infrastructure, frontières claires

Développer la partie front-end

App Router, Server Components, hooks personnalisés (use-real-time-chat...)

Développer la partie back-end

Route Handlers REST + SSE, Server Actions, validation Zod

Temps réel

Server-Sent Events + Upstash Redis (compatible serverless Vercel)

Preuves du dossier

20+ endpoints API

Architecture 4 couches

Push SSE temps réel

Server Actions



Préparer le déploiement d'une application sécurisée

BC03

Bloc de compétences

RNCP 37873 • CDA

En quoi le bloc consiste

Mettre en place une chaîne CI/CD complète, conteneuriser et déployer en environnement de production sécurisé.

Compétences couvertes par Konekt

Préparer et documenter le déploiement

Dockerfile multi-stage, docker-compose, variables d'env documentées

Contribuer à la mise en production DevOps

GitHub Actions : lint + test + build + deploy, secrets cloisonnés

Industrialiser la livraison

Vercel (preview par PR + prod), Neon branching pour BDD isolée par PR

Garantir l'exploitabilité

Healthcheck, rollback Vercel one-click, logs structurés, monitoring

Preuves du dossier

Docker multi-stage

CI/CD GitHub Actions

Vercel + Neon

Rollback + monitoring

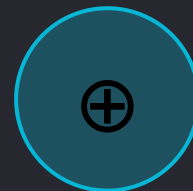


Difficultés rencontrées



Temps réel sans WebSocket persistant

Vercel serverless + Upstash Redis REST = pas de pub/sub TCP. J'ai construit un mécanisme SSE + polling Redis qui reste compatible serverless tout en garantissant l'ordre d'arrivée et la persistance Prisma avant push.



Modélisation Prisma de bout en bout

18 modèles avec des contraintes d'unicité, des cascades à choisir avec soin et des enums métier. J'ai itéré sur le MCD plusieurs fois avant d'arriver à un schéma équilibré.



Auth custom sans NextAuth

Choix assumé pour comprendre le fonctionnement du JWT. Il a fallu gérer le middleware, le state OAuth, l'injection de headers et l'éventuelle révocation — toujours en roadmap.



Documentation jury

Structurer le dossier de certification pour qu'il reste lisible par un jury non technique tout en étant suffisamment précis pour la partie technique. Beaucoup d'allers-retours sur la table des matières.

Ce que j'ai appris

Compétences techniques

- TypeScript strict, génériques, schémas Zod partagés
- Next.js 15 — App Router, Server Components, Actions
- Prisma — modélisation, migrations, requêtes complexes
- PostgreSQL — relations riches, contraintes, indexation
- Redis — cache + buffer SSE serverless-compatible
- Sécurité applicative — OWASP, RGPD, JWT, bcrypt
- Docker — multi-stage, compose, healthcheck, secrets
- CI/CD GitHub Actions — pipeline lint+test+build+deploy

Compétences transverses

- Découpe d'un gros projet en sprints + user stories
- Documentation technique adressée à des développeurs
- Lecture critique du code des coéquipiers (code review)
- Veille technique organisée et soutenable
- Estimation et priorisation pragmatique
- Justification de choix d'architecture face à un jury
- Communication métier ↔ technique (atout de reconversion)
- Endurance sur un projet long de plusieurs mois

Axes d'amélioration & roadmap

COURT TERME	MOYEN TERME	LONG TERME
- Sanitization markdown (DOMPurify) — XSS résiduelle ChatMessage - Rate limit /login, /register, /chat/send via @upstash/ratelimit - Headers HTTP de sécurité (CSP, X-Frame, Referrer) - Sentry + alertes prod	- Tests E2E Playwright sur les parcours critiques - Droit RGPD : suppression compte + export JSON - Notifications push natives (Web Push) - Modération & signalement	- Décomposer en services (chat → ws dédié) - Recommandations contenu personnalisées - App mobile dédiée (React Native ?) - Métriques produit + analytics

Conclusion

Konekt m'a permis de mettre en pratique l'ensemble du cycle de développement d'une application moderne — de la modélisation à la mise en production — tout en faisant des choix techniques justifiés et documentés.

Au-delà de la certification, ce projet matérialise ma transition concrète vers le métier de Concepteur Développeur d'Applications.



Cycle complet maîtrisé

De la conception à la production



Sécurité & RGPD pris au sérieux

Audit + roadmap durcissement



Choix techniques justifiés

Trade-offs assumés et documentés



Recul critique acquis

Limites identifiées, plan d'évolution

Disponible pour une alternance • 12 mois • Normandie / télétravail • CDA RNCP 37873

Konekt

Merci pour votre attention

Place aux questions

✉ djlike@hotmail.fr

🌐 clecart.fr

☎ 06 70 02 80 47

🔗 github.com/arocchet/social-network